ResolveFlow AI

Receiving-Institution Missing Incoming Payment Investigation & Reconciliation

Backend-only implementation specification | Version 1.0 | 9 November 2026

Purpose: build a portfolio-grade internal payment operations service inspired by a real Monzo-to-LenFi transfer that was reported successful by the sender, could not initially be located by the receiving institution, and was later credited after the sender supplied a payment reference.

Important: The actual cause of that delay was never disclosed. Every failure cause below is a synthetic scenario, not a claim about Monzo or LenFi. Neither institution is integrated with this project.

1. Problem definition and institutional ownership

ResolveFlow operates on behalf of the receiving institution (LenFi-like institution in the demonstration), not the sending bank. Its goal is to reduce manual tracing, fragmented support handoffs and unexplained delays when an incoming transfer is reported missing. It investigates the receiving institution's records and its authorised provider/rail data, then recommends or executes only policy-permitted actions.

Observed customer journey: (1) customer sends funds from Monzo to LenFi; (2) sending bank reports success; (3) recipient account shows no credit; (4) LenFi support cannot initially locate the payment; (5) customer obtains a reference from Monzo and shares it with LenFi; (6) funds appear days later without a cause being explained.

Desired future journey: customer complaint or automatic unmatched-payment alert -> structured investigation -> evidence-based status -> safe credit/allocation where authorised, or trace/escalation -> recorded resolution and communication-ready case summary.

2. System boundary and what it can actually know

System	Accessible information / limitation
Receiving institution	Own customer accounts, incoming ledger, allocations, unallocated receipts, support cases, internal restrictions.
Authorised payment provider / rail	Only status, settlement, return and reference data exposed by contracted APIs or authorised messages.
Sending bank	Not directly accessible. Customer-supplied reference is a lookup clue, not proof of settlement or permission to access Monzo systems.
External recipient-account state	Cannot be inferred from a sender-side success message. ResolveFlow distinguishes submission, acceptance, settlement, receipt and customer credit.

In the portfolio simulator, an internal receiving-bank ledger and a separate simulated payment-provider adapter generate controlled events. A simulated sending bank provides synthetic origin data only; the agent does not receive unrestricted access to it.

3. Primary functional requirements

- Accept missing incoming-payment reports containing receiving customer/account, amount, currency, expected date, sender institution and optional sender reference.
- Ingest signed/validated synthetic provider events, incoming bank records and customer-ledger postings; deduplicate repeated events.
- Find exact reference matches first; use amount, currency, dates and beneficiary identifiers only as supplementary evidence, not sole authority to credit.

- Reconcile provider receipt/settlement with internal allocations and account credits; identify unmatched receipts and conflicts.
- Produce a structured investigation report with observed facts, missing evidence, suspected cause (if supported), recommended action and evidence references.
- Run deterministic policy checks before any account credit, release or case closure; route ambiguous cases to human review.
- Support approval/rejection through protected backend endpoints and persist an audit trail.
- Proactively scan unmatched incoming receipts as well as customer-reported cases.

4. End-to-end workflow

Entry A - customer complaint: POST /cases with transaction reference and beneficiary details. Entry B - proactive: reconciliation job finds provider-confirmed receipt without matching customer credit.

Flow: intake -> validate/normalise -> collect internal ledger + provider evidence -> reconstruct lifecycle -> match beneficiary -> assess uncertainty/restrictions -> propose resolution -> policy engine -> [AUTO_ALLOCATE | WAIT_AND_RECHECK | HUMAN_REVIEW | TRACE_WITH_PROVIDER] -> post verified action if permitted -> audit + case status.

An agent may investigate and propose; the policy engine, not the LLM, owns authorisation. Before any posting, the transaction is re-read under a database transaction and checked for duplicate credits, beneficiary consistency, restrictions and available backing funds.

5. Core scenarios and expected outcomes

Scenario	Evidence	Correct behaviour
S1. Funds received, not allocated	Provider confirms funds received; unmatched receipt exists; beneficiary is uniquely verified; no prior credit.	Auto-allocate only if explicit institutional policy allows and all controls pass; otherwise request approval.
S2. Reference supplied, no receipt	Customer gives sending-bank reference; no matching authoritative provider receipt.	Do not credit. Create trace request, recheck later and escalate according to SLA.
S3. Ambiguous beneficiary	Provider receipt exists, but account identity conflicts or multiple possible matches.	Hold unallocated funds; require human investigation and approval after evidence is corrected.
S4. Duplicate event / replay	Same provider event delivered twice or credit already posted.	Idempotent no-op; no duplicate customer credit; audit duplicate.
S5. Conflicting statuses	Provider says received, internal ledger says reversed or restricted.	No auto-credit; human review and reconciliation of authoritative records.
S6. Late arrival	Initial status unknown; provider confirms receipt later.	Resume case, re-evaluate fresh evidence and policy, then allocate or escalate.

6. Decision and autonomy policy

Decision	Conditions	System action
AUTO_ALLOCATE	Confirmed provider receipt; verified beneficiary; funds backed by a posted unmatched receipt; no prior credit; no holds; policy explicitly permits.	Execute idempotent simulated ledger allocation; audit and close case.
WAIT_AND_RECHECK	No conclusive receipt or a pending provider status; still within defined retry/SLA window.	Persist waiting state and schedule bounded retry; never credit on customer assertion alone.
TRACE_WITH_PROVIDER	Payment reference exists but receiving institution lacks authoritative evidence.	Open simulated provider trace and attach request ID; notify operations if unresolved.
HUMAN_REVIEW	Beneficiary ambiguity, conflicting evidence, fraud/compliance hold, unsupported action or policy threshold exceeded.	Persist approval request; LangGraph interrupt; authenticated reviewer approves/rejects; revalidate before execution.

Sample simulation policy (not a universal banking rule): automatic allocation is permitted only for an exact, verified match to an already-received unallocated payment. The default is no automatic posting where any evidence or restriction check fails.

7. Agent design and scoped tools

Component	Responsibility / allowed tools
Payment Trace node	Read-only tools: get_case, get_internal_incoming_payments, get_ledger_entries, query_provider_status, get_event_history.
Reconciliation node	Deterministically compare authoritative receipt, beneficiary match and existing credit; output discrepancies and evidence IDs.
Investigation / explanation node	LLM produces evidence-grounded structured hypotheses and a human-readable explanation; cannot invent provider status.
Resolution proposal node	Proposes AUTO_ALLOCATE, WAIT_AND_RECHECK, TRACE_WITH_PROVIDER or HUMAN_REVIEW with reasons and evidence refs.
Policy + posting services	Pure deterministic validation, role checks, double-entry posting, uniqueness constraints, audit; not directly callable as unrestricted agent tools.

Use one persisted LangGraph workflow with specialised nodes rather than four independent autonomous agents in the three-day MVP. Persist case state and support interrupt/resume for human review. Model output is advisory; authoritative financial state comes from verified adapters and database records.

8. Backend services and data model

Entity	Minimum fields
customers / accounts	tenant_id, customer_id, account_id, currency, status
incoming_payments	payment_id, provider_reference, sender_bank, beneficiary_account_id, amount_minor, currency, provider_status, received_at
payment_events	event_id, source, payment_id, event_type, event_time, ingested_at, payload_hash
ledger_entries	journal_id, account_id, debit_minor, credit_minor, posted_at, source_payment_id
investigation_cases	case_id, customer_id, reference, state, findings_json, evidence_refs, proposed_action
approval_requests	approval_id, case_id, reviewer_id, decision, reason, decided_at
audit_events	actor, action, policy_version, before_state, after_state, timestamp

Store money as integer minor units; enforce balanced journals, unique event IDs, unique allocation per provider receipt, atomic transactions and tenant-scoped access. For a simulated allocation, debit the unallocated-receipts liability and credit the customer-deposit liability; do not manufacture new funds.

9. API contract (backend only)

Method + route	Purpose
POST /api/v1/cases	Create missing incoming-payment complaint
GET /api/v1/cases/{id}	Read case state, evidence and proposed action
POST /api/v1/provider-events	Ingest validated synthetic payment-provider event
POST /api/v1/investigations/{id}/run	Start or resume investigation
POST /api/v1/approvals/{id}/decision	Authorised approve/reject (role-scoped)
GET /api/v1/payments/{id}/trace	Return payment lifecycle and evidence timeline
POST /api/v1/reconciliation/run	Trigger synthetic unmatched-receipt scan
GET /api/v1/audit-events	Retrieve filtered audit records

Example complaint: {"beneficiary_account_id":"LEN-001","sender_bank":"Monzo","payment_reference":"MON-78291","amount_minor":50000,"currency":"GBP"}

Example result: {"case_id":"CASE-001","status":"awaiting_review","provider_receipt":"confirmed","customer_credit":"missing","proposed_action":"allocate_unmatched_receipt","execution_status":"not_executed"}. These are illustrative contracts, not implemented outputs.

10. Security, financial controls and failure handling

- Authentication and role-based authorisation for support staff, payment operations and approvers; never let a customer complaint directly trigger privileged posting.
- Provider adapters validate event authenticity and schema; a reference is not a substitute for authoritative settlement/receipt evidence.
- Enforce database transactions, unique source-payment allocation keys, idempotency tokens, journal balancing and race-condition tests.

- Separate investigator permissions (read/propose) from posting authority (deterministic service); approval must be bound to exact action and evidence version.
- After a human approval, re-read current payment status and policy to avoid stale approvals.
- Log every decision and evidence reference; redact personal identifiers in model prompts and observability traces.
- Bound retries, backoff, timeout and dead-letter handling for failed provider lookups; no silent assumption that timeout means nonreceipt.
- Use synthetic data only; no real Monzo/LenFi credentials, customer records or production banking access.

11. Evaluation and acceptance criteria

- All six synthetic scenarios produce the expected decision and evidence-backed case state.
- Zero duplicate credits across event replay, repeated approvals and concurrent allocation regression tests.
- Zero allocations when authoritative receipt is absent, beneficiary match is ambiguous or account is restricted.
- All simulated ledger journals balance; no posting occurs outside the deterministic service.
- Each case includes traceable evidence IDs, source timestamps, policy version and final action status.
- Human-review cases persist and can resume safely; rejected approvals cannot execute.
- Report case latency, tool failures, reconciliation precision and scenario pass rates as measured results, not assumed achievements.

12. Three-day backend delivery plan

Day	Deliverables	Exit criterion
Day 1 - foundation	FastAPI, PostgreSQL, SQLAlchemy/Alembic, simulated provider and receiving-bank ledger, incoming events, idempotency.	Can seed received/unreceived payments and query consistent state.
Day 2 - investigation	LangGraph trace/reconcile/propose workflow, deterministic policy engine, safe simulated allocation, case persistence, approval endpoint.	End-to-end S1-S3 work with auditable outcomes.
Day 3 - proof	Duplicate/late-event tests, concurrency regression, human interrupt/resume, Langfuse if time, README, OpenAPI demo and metrics.	S1-S6 pass; demo shows safe auto-resolution and blocked unsafe actions.

Explicitly deferred: real interbank connectivity, Kafka, production fraud ML, multimodal document verification, cloud Terraform, customer-facing UI and production deployment. These belong to later iterations, not the three-day MVP.

13. Demonstration and portfolio narrative

Demo 1: provider receipt arrives, internal customer credit missing, ResolveFlow traces and safely allocates after policy checks. Demo 2: sender reference supplied but no receipt, system refuses credit and opens provider trace. Demo 3: ambiguous beneficiary, graph pauses for operator decision. Demo 4: replay the same provider event and prove no duplicate credit.

Portfolio framing: "Designed a receiving-institution AI payment investigation backend that reconciles provider receipts against customer-ledger credits, proposes evidence-grounded resolutions, and enforces idempotent, policy-controlled financial actions with human oversight." Add measured test counts and latency only after implementation.

14. Assumptions, unknowns and source alignment

Known from user account: sending institution was Monzo; receiving institution was LenFi; sender reported success; LenFi initially could not find the transfer; a Monzo reference was shared; funds appeared days later; LenFi did not explain the cause.

Unknown: actual payment rail, LenFi's provider or sponsor-bank architecture, whether settlement occurred before the initial complaint, the real failure cause, and whether credit was manual or automatic. Do not assert these as facts.

Adaptation from original October 2026 blueprint: retains event-driven investigation, LangGraph, PostgreSQL, synthetic bank/payment simulation, evidence-based decisions, policy controls, double-entry ledger, idempotency, human review and evaluation. Narrows the product from general payment recovery/fraud investigation to receiving-side missing incoming payment reconciliation for a three-day backend-only portfolio MVP.

Design status: proposed specification, not implemented or validated against real institution APIs.